APPLICATION SECURITY ASSESSMENT

TenantDesk v0.1.0

Multi-tenant SaaS: REST API, MCP server and Stripe webhook

Prepared by **Azeem Haider**, Backend Integration Engineer
Sample report · Version 1.0

Sample report. TenantDesk is a demo application built for this portfolio, and the issues in v0.1.0 were introduced on purpose to show the assessment process end to end. No client systems or real customer data were involved. Code and reproduction scripts: github.com/Azeem-Haider/tenantdesk-mcp (tag v0.1.0 for the assessed build, main for the fixed build).

1. Executive summary

TenantDesk lets several clinics share one platform, with an AI assistant (via MCP) that reads and acts on their data. The assessment asked one question first: **can one clinic, or an AI acting for it, reach another clinic's data or money?**

In v0.1.0 the answer was **yes, in three separate ways**: a changed parameter in the customer export, an invoice lookup without a tenant check, and an AI tool that accepted a tenant id from the model. The Stripe webhook also accepted forged payment events from anyone on the internet, and the AI refund tool let any staff member refund any amount in a single call.

3	6	0	1
High	Medium	Low	Informational

Outcome: all 10 findings were fixed in v0.2.0 and every one was retested with the same proof-of-concept script, which now reports **0 of 10 vulnerable**. The fixes are structural (one tenant-scoped data layer, verified webhooks, two-step refunds), so the same classes of bug are unlikely to come back with new features.

2. Scope and rules of engagement

Target	TenantDesk v0.1.0 (git tag `v0.1.0`), run locally with seeded demo data
In scope	REST API (`/api/customers/export`, `/api/invoices/:id`); MCP server over Streamable HTTP and stdio (7 tools); Stripe webhook (`/webhooks/stripe`); authentication and authorization; data layer
Out of scope	Infrastructure, hosting and network; denial-of-service; third-party services (Stripe itself, the AI model provider); social engineering
Test accounts	Two tenants (Acme Dental, Brightside Clinic) with admin, agent and viewer users
Approach	Grey box: source code available, plus authenticated testing as each role
Data	Synthetic only. No production systems were touched.

3. Method

- **Threat model first:** for each entry point, who can call it, what identity it trusts, and what it can change.
- **Checklists:** OWASP API Security Top 10 (2023), OWASP Top 10 (2021) and selected OWASP ASVS 4.0 Level 2 controls for access control, authentication, error handling and logging.
- **AI/MCP-specific:** tool arguments treated as attacker-controlled (prompt injection), privilege of AI actions, human confirmation for money-moving tools, auditability.

- **Payments:** webhook authenticity, replay windows, duplicate delivery and idempotency.
- **Evidence:** every finding is backed by a scripted proof of concept (`assessment/poc.ts`) so it can be reproduced and retested exactly.

Severity: CVSS v3.1 base score, with business impact noted where it differs (for example TD-06, which moves money).

4. Findings summary

ID	FINDING	SEVERITY	CVSS	STATUS
TD-01	Cross-tenant customer export via tenant_id parameter	High	7.7	Fixed
TD-02	Insecure direct object reference on GET /api/invoices/:id	Medium	5.0	Fixed
TD-03	MCP tool accepts tenant_id from the model (prompt-injection data access)	High	7.7	Fixed
TD-04	Stripe webhook accepted without signature verification	High	7.5	Fixed
TD-05	Webhook events not de-duplicated; refunds double-counted on retry	Medium	5.3	Fixed
TD-06	Refund tool missing role check, amount cap and human confirmation	Medium	6.5	Fixed
TD-07	Role and tenant taken from token claims; downgraded users keep old rights	Medium	4.2	Fixed
TD-08	Database error details exposed to the MCP client	Medium	4.3	Fixed
TD-09	CSV export allows spreadsheet formula injection	Medium	5.4	Fixed
TD-10	No audit trail of actions taken through MCP tools	Informational	n/a	Fixed

5. Detailed findings

TD-01 Cross-tenant customer export via tenant_id parameter

Severity	High CVSS 7.7
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N`
Component	REST API · src/api.ts, src/repo.ts (listCustomers)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

The customer export endpoint accepts an optional `tenant_id` query parameter and passes it to the data layer, which uses it instead of the caller's own tenant. Any signed-in user can download the full customer list (names, emails, plans) of any other clinic by changing one parameter.

STEPS TO REPRODUCE

```
GET /api/customers/export?tenant_id=t_bright
Authorization: Bearer <token for u_ben, Acme Dental agent>
```

EVIDENCE

HTTP 200, CSV contains Brightside Clinic customers (e.g. "Sam Okafor").

IMPACT

Full cross-tenant disclosure of customer personal data. This is the most serious class of bug in a multi-tenant SaaS: one customer can read another customer's data, which is reportable under GDPR and usually ends contracts.

RECOMMENDATION

Never accept a tenant identifier from the request. Derive the tenant from the authenticated session only, and enforce it in the shared data layer so no endpoint can override it. Add Postgres row-level security as a second barrier.

RETEST

v0.2.0: the parameter is ignored; `TenantScope` always filters by the session tenant. Retest returned only Acme data.

TD-02 Insecure direct object reference on GET /api/invoices/:id

Severity	Medium CVSS 5.0
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:N/A:N`
Component	REST API · src/repo.ts (getInvoice)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

The invoice lookup queries by invoice id only, without a tenant filter. A user who knows or guesses another tenant's invoice id can read its amount, status and customer reference.

STEPS TO REPRODUCE

```
GET /api/invoices/inv_b1
Authorization: Bearer <token for u_ben, Acme Dental agent>
```

EVIDENCE

HTTP 200 with Brightside Clinic invoice inv_b1.

IMPACT

Cross-tenant disclosure of billing data. Invoice ids are short and sequential-looking, so enumeration is practical.

RECOMMENDATION

Scope every lookup by tenant (`where tenant_id = $session_tenant and id = $id`) and return the same 404 for "belongs to someone else" and "does not exist".

RETEST

v0.2.0: lookup is tenant-scoped; retest returned 404.

TD-03 MCP tool accepts tenant_id from the model (prompt-injection data access)

Severity	High CVSS 7.7
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N`
Component	MCP server · src/server.ts (list_customers)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

The `list_customers` tool exposes a `tenant_id` argument. The AI model fills tool arguments, so any text the model reads (a support ticket, an email, a document) can instruct it to pass another tenant's id. The server trusts the argument.

STEPS TO REPRODUCE

```
tools/call list_customers {"tenant_id": "t_bright"}
(as u_ben, Acme Dental agent, over the MCP HTTP endpoint)
```

EVIDENCE

Tool returned Brightside Clinic customers.

IMPACT

Same data exposure as TD-01, but reachable through prompt injection: an attacker never needs credentials, only a way to get text in front of the model.

RECOMMENDATION

MCP tools must not take tenant, user or role identifiers. Resolve the principal from the transport (verified token) and apply it server-side. Treat every tool argument as attacker-controlled.

RETEST

v0.2.0: no tool accepts a tenant id; extra arguments are dropped. Retest returned only Acme data.

TD-04 Stripe webhook accepted without signature verification

Severity	High CVSS 7.5
Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N`
Component	Webhook · src/api.ts (handleStripeWebhook)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

The `/webhooks/stripe` endpoint parses and acts on any JSON body. It does not check the `Stripe-Signature` header, so anyone on the internet can send fake payment events.

STEPS TO REPRODUCE

```
POST /webhooks/stripe
{"id":"evt_forged_1","type":"invoice.paid","data":{"object":{"metadata":
{"invoice_id":"inv_a2"}}}}
(no signature header, no authentication)
```

EVIDENCE

HTTP 200; open invoice inv_a2 changed to "paid".

IMPACT

Unauthenticated attackers can mark invoices as paid (getting service without paying) or trigger any other webhook-driven state change.

RECOMMENDATION

Verify the HMAC-SHA256 signature over the raw body with the endpoint secret, compare in constant time, and reject events outside a 5-minute timestamp window.

RETEST

v0.2.0: signature and timestamp verified; retest got HTTP 400 and the invoice stayed open.

TD-05 Webhook events not de-duplicated; refunds double-counted on retry

Severity	Medium CVSS 5.3
Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`
Component	Webhook · src/api.ts
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

Stripe delivers events at least once and retries on timeouts. The handler processes every delivery and adds the refunded amount each time instead of storing Stripe's running total.

STEPS TO REPRODUCE

```
Deliver the same signed charge.refunded event (amount_refunded: 2000) three times.
```

EVIDENCE

Invoice refunded_cents became 6000 instead of 2000.

IMPACT

Financial records drift from Stripe; refunds and revenue reports are wrong. Happens on its own during normal Stripe retries, and on purpose when combined with TD-04.

RECOMMENDATION

Record each event id in a processed-events table and skip duplicates. Write absolute values from the event (Stripe's totals), never increments.

RETEST

v0.2.0: events de-duplicated by id; refund total is set, not added. Retest: refunded_cents = 2000.

TD-06 Refund tool missing role check, amount cap and human confirmation

Severity	Medium CVSS 6.5
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`
Component	MCP server · src/server.ts, src/repo.ts (issue_refund)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

The `issue_refund` tool refunds immediately in one call, for any amount, for any signed-in role. The model can trigger it on its own, including after a prompt injection.

STEPS TO REPRODUCE

```
tools/call issue_refund {"invoice_id":"inv_a1","amount_cents":12000,"reason":"..."}
(as u_ben, an agent, not an admin)
```

EVIDENCE

Full $120.00 refunded by an agent with no confirmation step.

IMPACT

Direct money loss. Business impact is higher than the CVSS score suggests, because one manipulated conversation can issue refunds.

RECOMMENDATION

Split money-moving actions into request and confirm steps, restrict to admins, cap the amount the AI may start, bind the confirmation to the same user and a short expiry, and mark the tool as destructive so MCP clients ask the human.

RETEST

v0.2.0: request_refund + confirm_refund, admin only, $500 cap, same-admin, 10-minute expiry, single use. Retest: agent denied, nothing refunded.

TD-07

Role and tenant taken from token claims; downgraded users keep old rights

Severity	Medium CVSS 4.2
Vector	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N`
Component	Auth · src/auth.ts (principalFromToken)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

If a token carries `tenant_id` and `role` claims, they are trusted without checking the database. A user who is downgraded or removed keeps their old role until the token expires (8 hours).

STEPS TO REPRODUCE

```
Use a valid token minted with role=admin for u_cara, whose current role is viewer, and call create_ticket.
```

EVIDENCE

Ticket created by a user who is a viewer in the database.

IMPACT

Revocation doesn't take effect; a fired employee or downgraded user keeps access for hours.

RECOMMENDATION

Put only the user id in the token and load tenant and role from the database on each request (or keep tokens very short-lived with a revocation list).

RETEST

v0.2.0: claims ignored; role and tenant loaded per call. Retest: denied.

TD-08 Database error details exposed to the MCP client

Severity	Medium CVSS 4.3
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N`
Component	MCP server · src/server.ts (error handling)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

Unexpected errors are returned to the client verbatim, including raw Postgres messages.

STEPS TO REPRODUCE

```
tools/call search_invoices {"from":"2026-13-45"}
```

EVIDENCE

Response: `date/time field value out of range: "2026-13-45"`

IMPACT

Reveals the database engine and query structure, which helps an attacker craft further attacks; also puts internal details into AI conversation logs.

RECOMMENDATION

Return generic messages for unexpected errors, log the detail server-side with a correlation id, and validate inputs strictly before they reach the database.

RETEST

v0.2.0: unexpected errors return "internal error". Retest confirmed.

TD-09 CSV export allows spreadsheet formula injection

Severity	Medium CVSS 5.4
Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`
Component	REST API · src/api.ts (csvCell)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

Customer fields are written to CSV unchanged. A value starting with =, +, – or @ becomes a live formula when staff open the export in Excel or Google Sheets.

STEPS TO REPRODUCE

```
Set a customer name to =HYPERLINK("http://evil.test","Click") and export as an admin.
```

EVIDENCE

Exported cell contains the live formula.

IMPACT

Phishing links or data-exfiltration formulas run in staff spreadsheets.

RECOMMENDATION

Prefix cells that start with formula characters with a single quote and quote-escape every cell.

RETEST

v0.2.0: formula characters neutralised. Retest: exported as text.

TD-10 No audit trail of actions taken through MCP tools

Severity	Informational CVSS n/a
Vector	`Not scored`
Component	MCP server · src/repo.ts (audit)
Status	**Fixed in v0.2.0, verified on retest**

DESCRIPTION

Tool calls made by the AI, including refunds and data reads, are not recorded.

STEPS TO REPRODUCE

```
Run the checks above and inspect audit_log.
```

EVIDENCE

0 rows after 9 tool and API actions.

IMPACT

No way to investigate what the AI did, who asked it, or whether data was accessed after an incident.

RECOMMENDATION

Log every tool call with tenant, user, tool, arguments and outcome, including denied calls, and alert on spikes of denials.

RETEST

v0.2.0: every call logged, allowed or denied.

6. Retest results

The same script was run against the assessed build (v0.1.0) and the fixed build (v0.2.0).

ID	FINDING	V0.1.0	V0.2.0
TD-01	Cross-tenant customer export via tenant_id parameter	Vulnerable	Not vulnerable
TD-02	Insecure direct object reference on GET /api/invoices/:id	Vulnerable	Not vulnerable
TD-03	MCP tool accepts tenant_id from the model (prompt-injection data access)	Vulnerable	Not vulnerable
TD-04	Stripe webhook accepted without signature verification	Vulnerable	Not vulnerable
TD-05	Webhook events not de-duplicated; refunds double-counted on retry	Vulnerable	Not vulnerable

ID	FINDING	V0.1.0	V0.2.0
TD-06	Refund tool missing role check, amount cap and human confirmation	Vulnerable	Not vulnerable
TD-07	Role and tenant taken from token claims; downgraded users keep old rights	Vulnerable	Not vulnerable
TD-08	Database error details exposed to the MCP client	Vulnerable	Not vulnerable
TD-09	CSV export allows spreadsheet formula injection	Vulnerable	Not vulnerable
TD-10	No audit trail of actions taken through MCP tools	Vulnerable	Not vulnerable

7. Recommendations beyond the fixes

- **Defence in depth for tenancy:** add Postgres row-level security keyed on a per-request tenant setting, so a missed filter in code is still blocked by the database.
- **Cross-tenant tests in CI:** keep a test per endpoint and per MCP tool that tries another tenant's ids. They are cheap and catch regressions immediately.
- **MCP tool design rules:** no identity arguments, read and write tools separated, destructive tools marked and confirmed by a human, inputs validated by schema.
- **Webhook checklist:** verify signature on the raw body, enforce a timestamp window, de-duplicate by event id, write absolute values, respond fast and process asynchronously.
- **Monitoring:** alert on spikes of denied tool calls and cross-tenant 404s; both are early signs of probing or prompt injection.
- **Identity:** move from shared-secret JWTs to the company identity provider (OAuth 2.1 / OIDC) before production.

Appendix: reproduce this report

```
git clone https://github.com/Azeem-Haider/tenantdesk-mcp
cd tenantdesk-mcp && npm install
git checkout v0.1.0 && npx tsx assessment/poc.ts   # 10 of 10 vulnerable
git checkout main   && npx tsx assessment/poc.ts   # 0 of 10 vulnerable
```